

Trabajo con Fechas y Hora

LocalDate · LocalTime · LocalDateTime · ZonedDateTime

Paquete java.time · Java SE 21 / JDK 21

Índice de Contenidos

1. Tabla resumen de clases

2. Conceptos clave

Inmutabilidad

Thread-Safe

3. LocalDate

LocalDate.now()

LocalDate.of()

plusDays / plusMonths / plusYears / minusDays

isBefore / isAfter

isLeapYear()

4. LocalTime

LocalTime.now()

LocalTime.of()

plusHours / plusMinutes / minusHours / minusMinutes

isBefore / isAfter

5. LocalDateTime

LocalDateTime.now()

LocalDateTime.of()

toLocalDate() y toLocalTime()

Manipulación y comparación

6. ZonedDateTime

ZonedDateTime.now()

ZonedDateTime.of()

ZonedDateTime.now(ZonedDateTime)

withZoneSameInstant()

7. Formatear fechas: DateTimeFormatter

ofPattern(String pattern)

ofPattern con Locale

.format()

LocalDate.parse()

DateTimeParseException

8. Documentación del proyecto: Gestión Global de Eventos

Clase Evento

Clase Agenda
Clase OrdenadorEventos
Clase Principal

1. Tabla resumen de clases

Clase	¿Tiene fecha?	¿Tiene hora?	¿Tiene zona?	Propósito
<code>LocalDate</code>	Sí	No	No	Hechos del calendario (ej. Vacaciones).
<code>LocalTime</code>	No	Sí	No	Horarios recurrentes (ej. Apertura de tienda).
<code>LocalDateTime</code>	Sí	Sí	No	Descripción de eventos sin contexto global.
<code>ZonedDateTime</code>	Sí	Sí	Sí	Momentos exactos con reglas políticas/geográficas.

2. Conceptos clave

Inmutabilidad

Cuando creas un objeto de fecha/hora, no se puede cambiar. Si le sumas un día, Java no modifica el original, sino que crea y devuelve un objeto completamente nuevo.



Siempre guarda el resultado de cada operación en una nueva variable. El objeto original permanece inalterado.

Thread-Safe (Seguridad de hilos)

Gracias a la inmutabilidad, estos objetos no dan problemas cuando varios procesos (hilos) intentan acceder a ellos al mismo tiempo — algo que sí ocurría con las versiones anteriores a Java 8.

3. `LocalDate`

Representa una fecha concreta (año, mes y día) sin tener en cuenta la hora ni la zona horaria.

`LocalDate.now()`

Obtiene la fecha actual del sistema.

```
// Obtener la fecha actual del sistema
LocalDate hoy = LocalDate.now();
System.out.println("Fecha actual: " + hoy);
```

`LocalDate.of(año, mes, día)`

Crea una fecha específica. Ejemplo: `LocalDate.of(2026, 3, 20)`.

```
// Crear una fecha específica (Ej. Fecha de caducidad)
LocalDate fechaCaducidad = LocalDate.of(2026, 12, 31);
System.out.println("Fecha de caducidad: " + fechaCaducidad);
```

plusDays / plusMonths / plusYears / minusDays

Suma días, meses o años a la fecha, o resta días. Recuerda: como son inmutables, el resultado debe guardarse en una nueva variable.

```
// Sumar y restar tiempo.
// IMPORTANTE: Como son inmutables, el resultado debe guardarse en una nueva
// variable.
LocalDate proximaSemana = hoy.plusDays(7);
LocalDate elMesPasado = hoy.minusMonths(1);
System.out.println("Dentro de 7 días será: " + proximaSemana);
System.out.println("Hace un mes fue: " + elMesPasado);
// Demostración de inmutabilidad: la variable 'hoy' no ha cambiado
System.out.println("Comprobación: La variable 'hoy' sigue siendo: " + hoy);
```

isBefore(LocalDate) / isAfter(LocalDate)

Compara si una fecha es anterior o posterior a otra.

```
// Comprobar si una fecha es anterior o posterior a otra
if (hoy.isAfter(fechaCaducidad)) {
    System.out.println("El producto está caducado.");
} else {
    System.out.println("El producto está en buen estado.");
}
```

isLeapYear()

Devuelve un booleano indicando si el año es bisiesto.

```
// Comprobar si un año es bisiesto
System.out.println("¿Es bisiesto el año " + hoy.getYear() + "? " +
hoy.isLeapYear());
// Comprobando un año que sabemos que es bisiesto (2024)
LocalDate fechaBisiesta = LocalDate.of(2024, 1, 1);
System.out.println("¿Es bisiesto el año 2024? " + fechaBisiesta.isLeapYear());
```

> Código completo en el archivo adjunto (Pkg local_Date).

4. LocalTime

La clase `LocalTime` representa exclusivamente una hora en el reloj (horas, minutos, segundos y hasta nanosegundos). No sabe en qué día estamos ni en qué parte del mundo nos

encontramos; es simplemente una hora abstracta. Al igual que `LocalDate`, es inmutable y segura para hilos (thread-safe).

Ejemplos de uso:

- › Definir el horario de apertura y cierre de una tienda (ej. abre a las 09:00 y cierra a las 20:00).
- › Programar una alarma para que suene todos los días a la misma hora.
- › Establecer la hora a la que un sistema debe hacer una copia de seguridad automática de madrugada.

`LocalTime.now()`

Captura la hora exacta del sistema en el momento en que se ejecuta la línea de código.

```
// Obtener la hora actual del sistema
LocalTime ahora = LocalTime.now();
System.out.println("Hora actual: " + ahora);
```

`LocalTime.of(hora, minuto, [segundo])`

Permite crear una hora específica. Puedes pasarle solo hora y minuto, o añadirle también los segundos.

```
// Crear horas específicas (Ej. Horario de una tienda)
LocalTime horaApertura = LocalTime.of(9, 0); // 09:00
LocalTime horaCierre = LocalTime.of(20, 30); // 20:30
System.out.println("La tienda abre a las: " + horaApertura);
```

`plusHours` / `plusMinutes` / `minusHours` / `minusMinutes`

Suma horas o minutos a tu tiempo original (devolviendo un nuevo objeto), o resta horas y minutos.

```
// Sumar y restar tiempo.
// Recuerda: Se crea un nuevo objeto, el original no se modifica.
LocalTime descansoCafe = ahora.plusHours(2);
LocalTime alarmaAviso = horaCierre.minusMinutes(15);

System.out.println("Dentro de 2 horas será el descanso: " + descansoCafe);
System.out.println("La alarma de cierre sonará a las: " + alarmaAviso);
System.out.println("Comprobación: La variable 'ahora' sigue siendo: " + ahora);
```

`isBefore(LocalTime)` / `isAfter(LocalTime)`

Compara dos horas para saber cuál va antes o después en el reloj.

```
// Ideal para saber si una acción está dentro de un horario permitido
```

```
// Vamos a simular que un cliente intenta entrar ahora mismo
System.out.println("¿Puede entrar un cliente en este momento?");

if (ahora.isAfter(horaApertura) && ahora.isBefore(horaCierre)) {
    System.out.println("Sí, la tienda está abierta.");
} else {
    System.out.println("No, la tienda está cerrada.");
}

System.out.println("\n¿Es la hora de apertura anterior a la de cierre? " +
    horaApertura.isBefore(horaCierre));
```

 *Código completo en el archivo adjunto (Pkg local_Time).*

5. LocalDateTime

`LocalDateTime` une una fecha (año, mes, día) y una hora (hora, minuto, segundo, nanosegundo) en un solo objeto. Sigue siendo inmutable, seguro para hilos (thread-safe) y, muy importante, no tiene zona horaria. Es decir, representa un instante en el calendario y el reloj, pero no sabe si es en España, México o Japón.

Ejemplos de uso:

- › Guardar en una base de datos local el momento exacto en que un usuario se registró en la aplicación.
- › Programar la fecha y hora del próximo examen de Programación (ej. 20 de mayo de 2026 a las 08:30).
- › Registrar el momento exacto en el que se emite una factura.

`LocalDateTime.now()`

Obtiene la fecha y hora exacta del sistema en este instante.

```
// Obtener la fecha y hora actual
LocalDateTime ahora = LocalDateTime.now();
System.out.println("Instante actual: " + ahora);
```

`LocalDateTime.of(año, mes, día, hora, minuto)`

Crea un instante específico. Puedes pasarle los datos sueltos, o incluso pasarle un objeto `LocalDate` y un `LocalTime` que ya tuvieras creados.

```
// Crear una fecha y hora específica (Ej. Fecha del examen final)
LocalDateTime examenFinal = LocalDateTime.of(2026, Month.MAY, 20, 8, 30);
System.out.println("El examen final es el: " + examenFinal);

// También puedes crearlo uniendo un LocalDate y un LocalTime
LocalDate diaTutoria = LocalDate.of(2026, 4, 15);
```

```
LocalTime horaTutoria = LocalTime.of(17, 0);
LocalDateTime tutoria = LocalDateTime.of(diaTutoria, horaTutoria);
System.out.println("La tutoría será el: " + tutoria);
```

toLocalDate() y toLocalTime()

Sirven para "romper" el objeto en dos y quedarte solo con la parte de la fecha o solo con la parte de la hora.

```
// Extraer solo la fecha o solo la hora de un LocalDateTime
LocalDate soloFechaExamen = examenFinal.toLocalDate();
LocalTime soloHoraExamen = examenFinal.toLocalTime();
System.out.println("Fecha del examen: " + soloFechaExamen);
System.out.println("Hora del examen: " + soloHoraExamen);
```

Métodos de manipulación y comparación

Tiene todos los plus...(), minus...(), isBefore() e isAfter() que ya hemos visto, pudiendo sumar o restar tanto días como horas.

```
// Podemos manipular tanto la parte de la fecha como la de la hora
LocalDateTime repasoGeneral = examenFinal.minusDays(2).minusHours(1);
System.out.println("El repaso general será: " + repasoGeneral);
System.out.println("Comprobación: La fecha del examen no ha cambiado: " +
examenFinal);

System.out.println("\n--- 4. COMPARACIÓN ---");
// Validar qué evento ocurre primero
if (tutoria.isBefore(examenFinal)) {
    System.out.println("Primero es la tutoría y luego el examen.");
} else {
    System.out.println("El examen es antes que la tutoría.");
}
```



Código completo en el archivo adjunto (Pkg local_Date_Time).

6. ZonedDateTime

Imagina que coges un LocalDateTime (que ya tiene fecha y hora) y le pegas una etiqueta que dice "Europa/Madrid" o "America/New_York". Eso es un ZonedDateTime. Representa un instante exacto y absoluto en la línea de tiempo global.



ZonedDateTime maneja automáticamente los cambios de hora (horario de verano e invierno). Si le dices "Europa/Madrid", Java ya sabe qué días del año hay que adelantar o atrasar el reloj.

Ejemplos de uso:

- › Vuelos internacionales: un avión sale de Madrid a las 10:00 (hora local) y el vuelo dura 8 horas hasta Nueva York.
- › Videollamadas globales: organizar una reunión a las 16:00 en España y saber qué hora es para un compañero en Japón.
- › Logs de servidores: empresas como Google o Amazon guardan a qué hora ocurrió un error con su zona horaria.

ZonedDateTime.now()

Obtiene la fecha, hora y zona actual de la máquina donde se ejecuta el código.

```
// ZonedDateTime.now()
ZonedDateTime ahoraAqui = ZonedDateTime.now();
System.out.println("Momento actual en mi zona: " + ahoraAqui);
```

ZoneId.of("Continente/Ciudad")

La forma de crear identificadores de zona válidos (ej. "Europe/Madrid", "Asia/Tokyo").

```
ZoneId zonaTokio = ZoneId.of("Asia/Tokyo");
```

ZonedDateTime.now(ZoneId zone)

Obtiene el instante actual, pero visto desde la zona horaria que le pidas.

```
// ZoneId.of("Continente/Ciudad") + ZonedDateTime.now(ZoneId)
ZoneId zonaTokio = ZoneId.of("Asia/Tokyo");
ZonedDateTime ahoraEnTokio = ZonedDateTime.now(zonaTokio);
System.out.println("Ahora mismo en Tokio son las: " + ahoraEnTokio);
```

withZoneSameInstant(ZoneId zone)

El más útil de todos: convierte una fecha/hora de una zona a otra, calculando la diferencia horaria automáticamente y manteniendo el mismo instante absoluto en el tiempo.

```
// withZoneSameInstant(ZoneId zone)
// Partimos de la hora que es AHORA AQUÍ, y calculamos qué hora es en Nueva York
ZoneId zonaNuevaYork = ZoneId.of("America/New_York");
ZonedDateTime ahoraEnNuevaYork = ahoraAqui.withZoneSameInstant(zonaNuevaYork);
System.out.println("Si llamo por videollamada AHORA MISMO a Nueva York, allí son las: " + ahoraEnNuevaYork);
```



Código completo en el archivo adjunto (Pkg zoned_Date_Time).

7. Formatear fechas: DateTimeFormatter

[`java.time.format`](#) (Desde aquí se puede acceder a todo lo que explicaré)

La clase `DateTimeFormatter` sirve para traducir las fechas entre el lenguaje que entiende Java y el lenguaje que entienden los usuarios. Se utiliza para convertir objetos de fecha y hora a texto (formatear) y viceversa (parsear). Al igual que el resto de las clases del paquete `java.time`, es inmutable y segura para hilos (thread-safe). Es la herramienta definitiva para que una aplicación de consola sea robusta y amigable.

Ejemplos de uso:

- › Pedir a un usuario que introduzca la fecha de un evento por teclado siguiendo un formato familiar (ej. 20/05/2026).
- › Mostrar información en pantalla utilizando el idioma local, traduciendo los números a texto (ej. "viernes, 17 de abril").
- › Procesar archivos CSV exportados de Excel donde las fechas vienen como cadenas de texto.

`DateTimeFormatter.ofPattern(String pattern)`

Crea un "molde" o patrón para establecer cómo se debe leer o escribir la fecha. Las letras definen el tipo de dato: dd (día de dos cifras), MM (mes en número), yyyy (año de cuatro cifras). También permite formatos textuales avanzados como MMMM (nombre completo del mes) o EEEE (nombre del día de la semana).

```
LocalDate hoy = LocalDate.now();
System.out.println("Fecha estándar de Java (sin formatear): " + hoy);

// Creamos el molde numérico tradicional español
DateTimeFormatter formatoEspanol = DateTimeFormatter.ofPattern("dd/MM/yyyy");

// Aplicamos el molde a nuestra fecha actual
String fechaBonita = hoy.format(formatoEspanol);
System.out.println("Fecha formateada (dd/MM/yyyy): " + fechaBonita);
```

`DateTimeFormatter.ofPattern(String pattern, Locale locale)`

Crea un patrón forzando una configuración regional concreta. Es fundamental cuando se usan formatos textuales completos (como "abril" o "lunes") para garantizar que el texto se genere en el idioma correcto, sin importar el idioma del sistema donde se ejecute el programa.

```
// Creamos un molde más avanzado que incluye los días de la semana y meses en
texto.
// Usamos Locale.of("es", "ES") para garantizar que las palabras salgan en
español.
DateTimeFormatter formatoElegante = DateTimeFormatter.ofPattern(
    "EEEE, d 'de' MMMM 'de' yyyy", Locale.of("es", "ES"));

System.out.println("Fecha elegante: " + hoy.format(formatoElegante));
```

.format(DateTimeFormatter formatter)

Este método se llama desde el propio objeto de fecha. Convierte el instante en un texto (String) agradable a la vista, aplicando el patrón que hemos diseñado previamente.

```

DateTimeFormatter formatoElegante = DateTimeFormatter.ofPattern(
    "EEEE, d 'de' MMMM 'de' yyyy", Locale.of("es", "ES"));

System.out.println("Fecha elegante: " + hoy.format(formatoElegante));

```

LocalDate.parse(CharSequence text, DateTimeFormatter formatter)

Realiza el proceso inverso y es vital para la entrada de datos. Toma un texto introducido por el usuario (un String) y el molde que hemos definido, y los transforma en un objeto LocalDate real que Java puede guardar, sumar, restar o comparar.

```

System.out.println("Vamos a registrar la fecha de un examen.");
// El bucle se repetirá hasta que el usuario introduzca el formato correcto
while (!fechaCorrecta) {
    System.out.print("Introduce la fecha del examen (dd/MM/yyyy): ");
    String textoUsuario = Leer.dato();
    try {
        // Intentamos encajar el texto del usuario en nuestro molde español
        fechaParseada = LocalDate.parse(textoUsuario, formatoEspanol);
        fechaCorrecta = true; // Si esta línea se ejecuta, no hubo errores

        System.out.println(";Lectura exitosa!");
        System.out.println("El examen ha sido registrado para el mes: " +
            fechaParseada.getMonth());
    }
}

```

Excepción: DateTimeParseException

Concepto clave en la lectura de datos. Si usamos el método .parse() y el usuario teclea algo que no coincide perfectamente con nuestro patrón (por ejemplo, escribe "20-05-2026" pero el patrón exigía barras "20/05/2026"), Java lanzará este error. Para evitar que el programa colapse, la lectura del teclado siempre debe envolverse en un bloque try-catch.

```

    } catch (DateTimeParseException e) {
        // Si salta la excepción, capturamos el error y avisamos al usuario
        System.out.println("ERROR: El texto '" + textoUsuario + "' no tiene el
formato correcto.");
        System.out.println("Por favor, asegúrate de usar las barras '/' y
números válidos.\n");
    }
}

```

8. Documentación del proyecto: Gestión Global de Eventos

Este proyecto presenta una solución robusta para la gestión de calendarios internacionales, diseñada específicamente para demostrar el potencial de la API de fechas de Java introducida

a partir de la versión 8. A diferencia de las herramientas antiguas de Java, este sistema garantiza precisión absoluta al trabajar con diferentes husos horarios.

Clase Evento

En esta clase se aplican los conceptos de inmutabilidad y especialización. En lugar de usar una sola variable para todo, cada evento desglosa el tiempo en:

- › `LocalDate`: Para la fecha pura (día/mes/año).
- › `LocalTime`: Para la hora exacta del reloj.
- › `LocalDateTime`: Para la combinación de ambas sin ubicación geográfica.
- › `ZonedDateTime`: El "instante global" que vincula el evento a una zona del mundo (ej: Asia/Tokyo).

Clase Agenda

Esta clase actúa como el motor de almacenamiento. Utiliza un `ArrayList` y se encarga de que la información esté organizada. Sus funciones principales son:

- › Recorrido con `Iterator`: Se utiliza un patrón para listar los eventos sin exponer la lista interna.
- › Control de Duplicados: Asegura que no existan dos eventos con el mismo nombre.
- › Búsqueda: Implementa métodos que devuelven el objeto encontrado o `null`.

Clase OrdenadorEventos

Para demostrar la flexibilidad del sistema, se incluye una clase dedicada exclusivamente a la ordenación personalizada. Permite organizar toda la agenda en orden alfabético descendente (de la Z a la A) mediante la interfaz `Comparator`.

Clase Principal

Presenta un menú interactivo donde se pueden ver todas las funcionalidades del proyecto en acción.